

AI SD

WYKŁAD XXVI

Algorytm Strassen

Dane: A, B - macierze $n \times n$ nad pierścieniem (?) R

Zadanie: Obliczyć $C = A \cdot B$

Klasycanie: $\Theta(n^3)$ lub (jako funkcja od rozmiaru danych - macierzy $\Theta(N^{\frac{3}{2}})$, gdzie $N = n \cdot n$ to rozmiar macierzy)

Dolna granica - $\Omega(N)$ - bo trzeba wypisać wynik

Algorytm Strassen:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$C_{11} = A_{11} \cdot B_{11} + A_{12} B_{21}$$

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

C_{12}, C_{21}, C_{22} - analogicznie (każde wymaga 2 mnożeń)

$$T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n) \Rightarrow \Theta(n^{\log_2 8}) = \Theta(n^3) \therefore$$

Rozwiazanie

$$m_1 = (A_{12} + A_{22}) \cdot (B_{21} + B_{22})$$

$$m_2 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$m_3 = (A_{11} + A_{21}) \cdot (B_{11} + B_{12})$$

$$m_4 = (A_{11} + A_{12}) \cdot B_{22}$$

$$m_5 = A_{11} (B_{12} - B_{22})$$

$$m_6 = A_{22} (B_{21} - B_{22})$$

$$m_7 = (A_{21} + A_{22}) B_{11}$$

$$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$$

$$T(n) = \Theta(n^{\log_2 7})$$

$$C_{11} = m_1 + m_2 - m_4 + m_6$$

$$C_{12} = m_4 + m_5$$

$$C_{21} = m_6 + m_7$$

$$C_{22} = m_2 - m_3 + m_5 - m_7$$

Wszystkie operacje na 4 częściach nie da się zejść
poniżej 4 mnożeń

Szybsze algorytmy:

→ Viktor Pan $\Theta(n^{\log_2 143640}) \approx n^{2.723}$ podzielił na 70

→ $\Theta(n^{2.3716})$ → uniwersalnie $\Theta(n^w)$ - oznacza złożoność
mnożenia macierzy

Domknięcie transytywne:

A - macierz sąsiedztwa

($A_{ij} = 1$ gdy i, j są 4 relacji)

G14

$$2 \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}^2 = A \cdot A^2 \cdot A^4 \dots A^n$$

złożoność $\Theta(\lg n \cdot n^w)$

Mnożenie macierzy logicznych

Adaptacja Strassen:

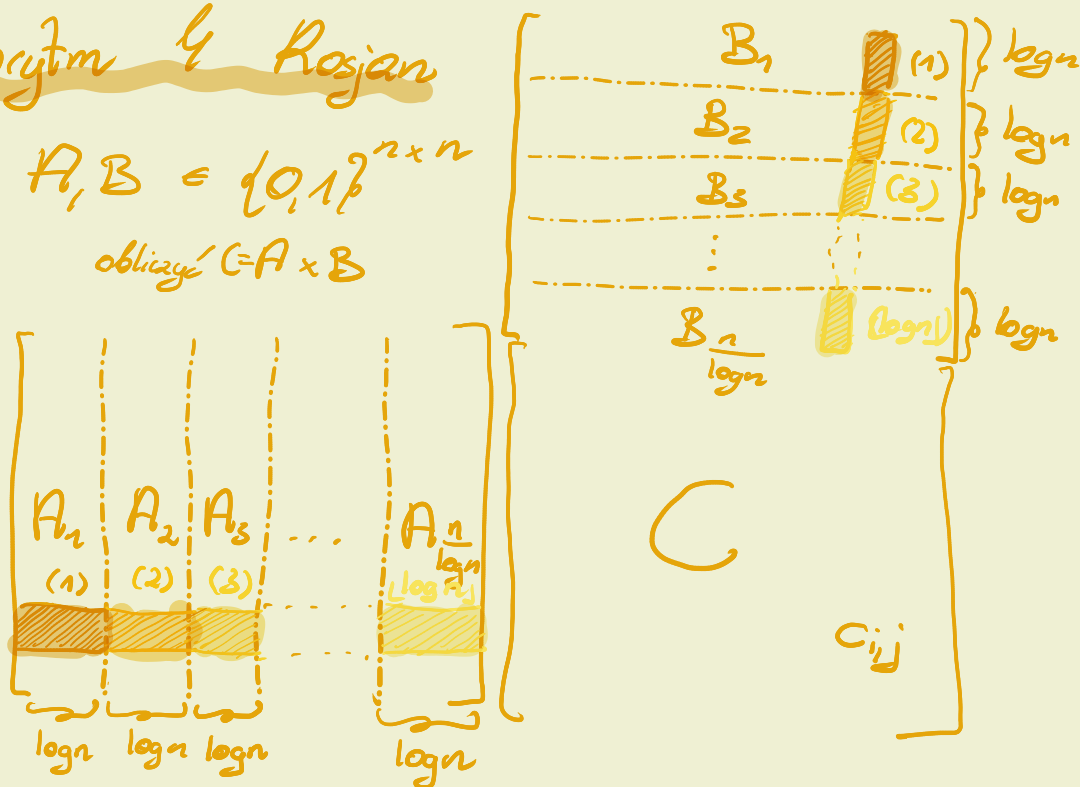
→ wprost nie można - bo nie ma odpowiednika odejmowania

→ true - 1 działania wykonujemy mod(n+1)
false - 0

Algorytm 4 Rosjan

dane: $A, B \in \{0,1\}^{n \times n}$

zadanie: obliczyć $C = A \times B$



$$C = \sum_{i=1}^{\frac{n}{2}} \begin{bmatrix} A_i \\ C_i \end{bmatrix} \begin{bmatrix} B_i \\ C_i \end{bmatrix} \quad c_{ij}^{(1)} = a_{i1} \cdot b_{1j} + a_{i2} \cdot b_{2j} + \dots + a_{i \frac{n}{2}} \cdot b_{\frac{n}{2}j}$$

$$c_{ij} = \underbrace{a_{i1} b_{1j} + \dots + a_{i \frac{n}{2}} b_{\frac{n}{2}j}}_{c_{ij}^{(1)}} + \underbrace{a_{i \frac{n}{2}+1} b_{1j} + \dots + a_{i \frac{3n}{4}} b_{\frac{n}{2}j}}_{c_{ij}^{(2)}} + \dots$$
$$= \sum_{k=1}^{\frac{n}{2}} c_{ij}^{(k)}$$

Podobna idea jest przy implementacji przybranego czołowi mnożenia macierzy.

Jak obliczyć C_i ?

1. Naivnie - zwykłym mnożeniem: C_i policzymy w $\Theta(n \cdot \log n \cdot n) = \Theta(n^2 \log n)$, ale takich C_i jest $\frac{n}{\log n}$, co daje $\Theta(n^2 \log n \cdot \frac{n}{\log n}) = \Theta(n^3)$ i

2. Obliczenie C_i realizujemy następująco:

1^o Zauważymy, że wszystkich możliwych wierszy A_i jest niewiele (tj. $2^{\log_2 n} = n$), dlatego też bardziej opłaca nam się przechować wszystkie możliwe iloczyny

$$\bar{a} \cdot B_i, \text{ gdzie } \bar{a} \in \{0, 1\}^{\log_2 n}$$

$$1 \left[\begin{array}{c} \bar{a} \\ \log_2 n \end{array} \right] \left[\begin{array}{c} B_i \\ \log_2 n \end{array} \right] \text{ obliczamy }$$

Najmniej $n \log n$, ale można to przyspieszyć dodając wiersz a B_i do któregoś z iloczynów

$$\begin{array}{cccc} [00 \dots 00] & [00 \dots 00] & [00 \dots 01] & [\text{do } 0 \text{ dodajemy ostatni} \\ & & & \text{wiersz}] \\ & [\text{---}] & & \\ & \uparrow & & \\ & \text{j-ty bit} & & \\ & \text{zapalony w } \bar{a} & & \end{array}$$

← j-ty wiersz B_i należy dodać do jednego z wyliczonych wyników (np. takiego, który od $\bar{a}$ różni się jedynie j-tym bitem)

To daje $\Theta(n)$ na 1 cache

Stąd $\Theta(n^2)$ przechowanie $\forall n$ możliwych wyników jest możliwe w $\Theta(n^2)$, no i ściąganie z cache raczej też.

Ale takich operacji wykonamy $\lceil \frac{n}{\log n} \rceil$ co daje złożoność $\Theta(\frac{n^3}{\log n})$.
No i ciężko to polepszyć bo samo dodawanie C_i zajmie $\Theta(\frac{n}{\log n} \cdot n^2)$, bo $\frac{n}{\log n}$ dodajemy macierze $n \times n$ (chyba, że jakiś multiplier surrender by pomógł). Zytania (zapisane z wykładu):

Zadanie 1:

jak pozbudować się $\log n$?

Spróbowując obliczenia tranzystornego do MM mamy złożoność $\log n \cdot n^w$

Zadanie 2:

jak zredukować mnożenie macierzy do domknięcia tranzystornego?

$$\begin{array}{c} 3n \\ \left[\begin{array}{ccc} I & A & 0 \\ 0 & I & B \\ 0 & 0 & I \end{array} \right] \\ \text{" } \\ 6 \end{array} \Rightarrow^* \left[\begin{array}{c} AB \\ \\ 6^* \end{array} \right]$$

Protocaniki (idea 2 2^{k+1} zamiast 2^k)

$$e_2: 2e_2 \equiv 1 \pmod{N}$$

$N = 17$ (nieparzyste)

Tworzymy grupy
 $G_0, G_1, G_2, \dots$

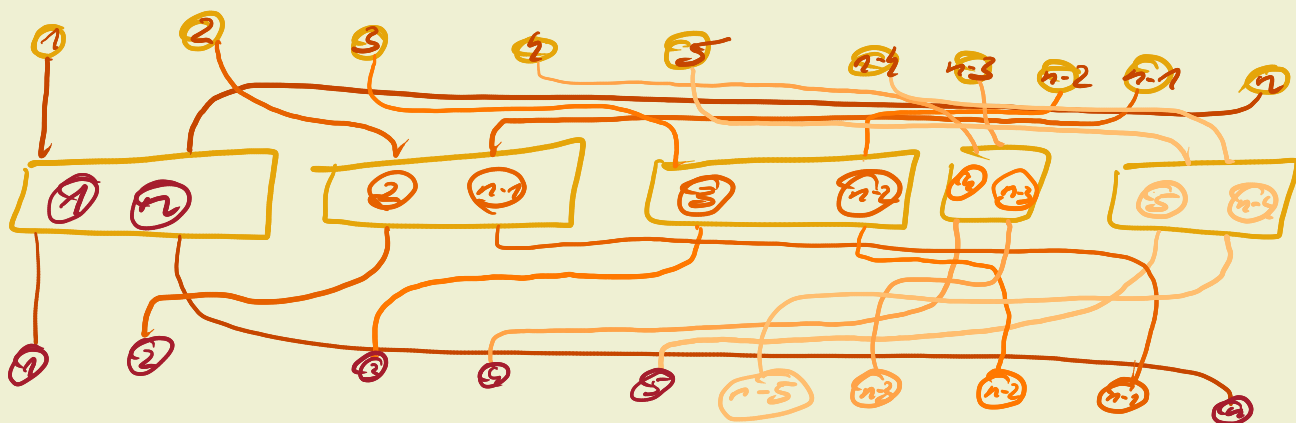
$$\rightarrow g \in G_i \Rightarrow 2g \in G_i$$

$$G_0 = \{0\}$$

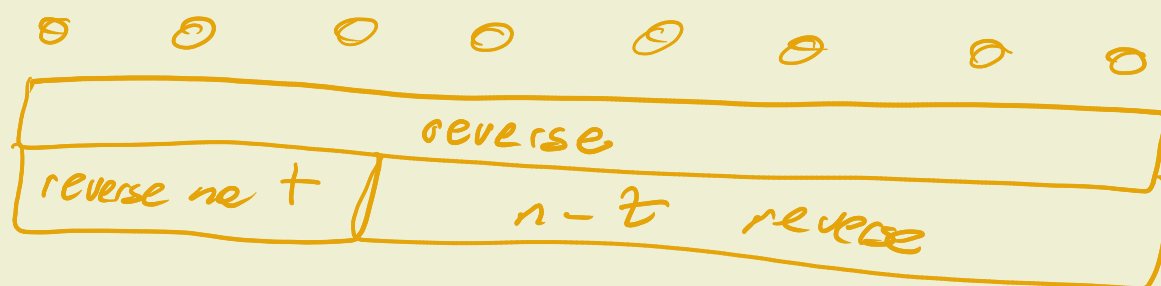
$$G_1 = \{1, 2, 4\}$$

$$G_2 = \{3, 6, 5\}$$

reverse:



shift (o jeden/słabo)



$ordi(x)$ definiujemy:

$$x = 2^{ordi(x)}$$

$$g_i \pmod{N}$$

$rev_i(x)$:

representant G_i

$$rev_i(x) = g_i \cdot 2^{-ordi(x)} \pmod{N}$$